

GPU-Accelerated Block Ciphers: Performance Analysis of ARX, Feistel, and SPN Architectures

Armaan Singh, Syam Sai Santosh, Bhargav Raman

March 2026

Abstract

Modern cryptographic systems must handle large volumes of data efficiently. Block ciphers, which operate on fixed-size data blocks, are naturally parallelizable since independent blocks can be encrypted simultaneously. This makes them well-suited for acceleration using Graphics Processing Units (GPUs). This project implements, optimizes, and benchmarks twelve symmetric ciphers across ARX, Feistel, and Substitution-Permutation Network (SPN) architectures. We evaluate each cipher across multiple implementation tiers—ranging from single-threaded baselines to heavily optimized CUDA GPU kernels—to analyze performance bottlenecks, memory transfer overheads, and the architectural factors that influence GPU speedup.

1 Introduction & Problem Statement

The goal of this project is to design and evaluate high-performance block cipher encryption systems across CPU and GPU architectures. Given a batch of plaintext blocks $P_1, P_2, \dots, P_N$, the system computes ciphertext blocks $C_i = E_k(P_i)$.

While Central Processing Units (CPUs) excel at sequential, complex control flows, GPUs provide massive data parallelism. By accelerating these ciphers, we aim to measure the practical speedup (T_{CPU}/T_{GPU}) and identify hardware bottlenecks such as PCIe memory transfer saturation. The project evaluates execution time, throughput (MB/s), and scalability across both Electronic Codebook (ECB) and Counter (CTR) modes.

2 Cryptographic Setup

Our evaluation covers twelve block ciphers, broadly categorized by their core design paradigms.

2.1 ARX-Based Ciphers

- **SPECK 128/128:** A lightweight ARX-based Feistel cipher with 32 rounds, 128-bit block and key.
- **LEA-128:** A Generalized Feistel Network (GFN) using 32-bit ARX operations across 4 branches, 24 rounds.
- **CHAM-128/128:** A lightweight ARX cipher optimized for constrained devices, 80 rounds with compact key schedule.
- **Threefish-256:** A pure ARX cipher (no S-boxes) used in the Skein hash function, 256-bit block, 72 rounds.

2.2 Lightweight Feistel Ciphers

- **SIMECK:** ARX-based Feistel structure using bitwise rotation, AND, and XOR operations.
- **SIMON:** Similar to SIMECK, using bitwise rotation-AND-XOR in a Feistel network.
- **TWINE:** A lightweight generalized Feistel cipher operating on 4-bit nibbles.
- **WARP:** A lightweight substitution-permutation cipher with round constants and linear diffusion.

2.3 Standard and Advanced SPN Ciphers

- **AES-128:** Standard 128-bit block size, 10 rounds, byte-oriented column-major state.
- **GIFT-64-128:** 64-bit block size, 28 rounds, packed 64-bit integer state.
- **SKINNY-64-128:** 64-bit block size, 36 rounds, row-decomposed linear layer.
- **PRESENT-128:** 64-bit block size, 31 rounds, bit-oriented permutation layer.

3 CPU Implementation

The CPU implementations serve as the baseline for correctness and performance benchmarking. Due to the wide variety of ciphers, multiple CPU optimization strategies were employed.

3.1 Single-Threaded and Python Reference Baselines

The naive implementations process one block at a time. For ARX ciphers (SPECK, LEA, etc.), pure Python implementations were batched to minimize parsing overhead. For Feistel ciphers (SIMECK, SIMON, TWINE, WARP), correct reference implementations were developed in C/C++ based on standard specifications. These serve as a correctness reference and absolute baseline.

3.2 Multi-Threaded and Compiled CPU Execution

To provide a competitive CPU benchmark, we applied two distinct parallelization strategies depending on the cipher stack:

- **Numba JIT (Python):** Used for AES, GIFT, SKINNY, and PRESENT. Python loops are compiled into native LLVM code. Multi-threading is achieved via OpenMP `prange` parallel block processing across 8 CPU threads.
- **Numba JIT (Python):** Used for ARX ciphers (SPECK, LEA, CHAM, Threefish). NumPy arrays with `@jit` compiled loops and CUDA event timing.
- **OpenMP (C/C++):** Used for SIMECK, SIMON, TWINE, and WARP. Encryption is parallelized across plaintext blocks using `#pragma omp parallel for`.

3.3 Bit-Slicing Optimization

For TWINE, WARP, SKINNY, and PRESENT, bit-sliced implementations were utilized on the CPU. By storing bits across a 32-bit or 64-bit word, multiple plaintext blocks (e.g., 32 blocks) are processed simultaneously. This enables SIMD-style parallelism on the CPU, heavily reducing memory overhead.

4 GPU Implementation (CUDA)

4.1 Parallelization Strategy and Thread Mapping

The GPU implementations leverage grid-stride loops to handle arbitrarily large inputs cleanly:

```
for (int i = idx; i < N; i += stride)
    encrypt(...)
```

Thread mapping varies based on the arithmetic intensity of the cipher:

- **1 Block/Thread:** Used for standard implementations (SIMECK, SIMON, SPECK, LEA, CHAM).
- **1 Batch of 32 Blocks/Thread:** Used for bit-sliced implementations (TWINE, WARP).
- **Coarsened Threads (2 to 8 Blocks/Thread):** Used for AES (2×), GIFT/SKINNY (4×), and PRESENT (8×) to amortize shared-memory setup overhead and hide global-memory latency.
- **Grid Size:** Along with Thread Coarsening, we use grid sizes of $\frac{N}{256 \times 2}$ for AES, $\frac{N}{256 \times 4}$ for GIFT and SKINNY, and $\frac{N}{\text{Block Size} \times 8}$ for PRESENT.

Standard block sizes of 256 threads were chosen to align with warp scheduling, ensuring high occupancy.

4.2 Memory Management

Efficient memory placement is critical for GPU throughput:

- **Global Memory:** Used for plaintext/ciphertext buffers (often ≥ 1 GB) which is use ful only for large datasets.
- **Shared Memory:** Used for AES S-boxes (256 B) to eliminate 160 global memory lookups per block. Also used to stage round keys for GIFT, SKINNY, and PRESENT.
- **Constant Memory:** Used for fast, read-only broadcast access. Round constants, lightweight S-boxes (16 B), and permutation tables utilize this tier.
- **Registers:** Temporary variables during encryption rounds are saved in these
- Device to Host and Host to Device transfers were measured

4.3 S-Box and Linear Layer Optimizations

Specific cipher optimizations include:

- **AES MixColumns:** Branchless `xtime` functions using compound-XOR tricks to avoid divergent warps.
- **PRESENT pLayer:** Decomposed a traditional 63-iteration bit loop into 4 delta-swap operations (20 instructions).
- **Bitsliced Boolean S-boxes:** Zero-memory-latency logic gate implementations for SKINNY and PRESENT, drastically outperforming constant-memory table lookups.
- **GIFT Fused SP Table:** A 256-entry constant-memory scatter table replaces 80 loop iterations (16 S-box + 64 P-box) with 16 table lookups.
- **SKINNY SBOX8:** Dual-nibble 8-bit table halves S-box memory operations (8 lookups vs. 16).

4.4 Mode of Operation: ECB vs. CTR

While ECB allows independent parallel encryption, CTR mode introduces counter construction and XOR overhead. For PRESENT, a dedicated **Native CTR Kernel** was developed that fuses counter generation, encryption, and the XOR step entirely on-device, bypassing host-side XOR bottlenecks.

5 Correctness Verification

Correctness was strictly verified across all tiers:

- Comparing GPU outputs byte-for-byte with OpenMP and JIT CPU outputs.
- Matching outputs against known-answer tests (NIST FIPS-197 for AES, published vectors for GIFT, SKINNY, PRESENT).
- Cross-implementation consistency checks: single-threaded, OpenMP, Numba JIT, and CUDA outputs were compared for identical inputs.
- Validating consistency across deterministic plaintext patterns with fixed keys.

Any mismatch in the pipelines immediately aborted the benchmarking process.

6 Performance Evaluation

6.1 Experimental Setup

Given the varied nature of the ciphers, benchmarks were conducted across multiple environments:

- **System A (ARX Benchmarks):** Intel Ultra 7 155H, NVIDIA RTX 4060 Laptop GPU (65W), 6GB VRAM, 16 GB RAM, CUDA 11.8.
- **System B (Lightweight/SPN Benchmarks):** AMD Ryzen 7 5800H, NVIDIA RTX 3050 Ti (95W), CUDA 12.3.
- **System C (Standard and Advanced SPN Ciphers):** Intel i7 11370H, NVIDIA RTX 3050 Ti (50W) 4GB VRAM, 16GB RAM, CUDA 12.8.

Input sizes swept Nine orders of magnitude: from $N = 2^{10}$ (1 KB) up to 10^8 blocks (~ 1.6 GB).

6.2 ARX Cipher Results (SPECK, LEA, CHAM, Threefish)

6.2.1 Execution Time vs. Input Size

Figure 1 shows execution time across all four implementation tiers (Naive, Optimized, Numba JIT, GPU) for the ARX ciphers. Naive and Optimized implementations scale linearly and are 2–3 orders of magnitude slower than JIT/GPU approaches at 1 MB. GPU execution time grows very slowly with input size, dominated by memory transfer overhead at smaller sizes.

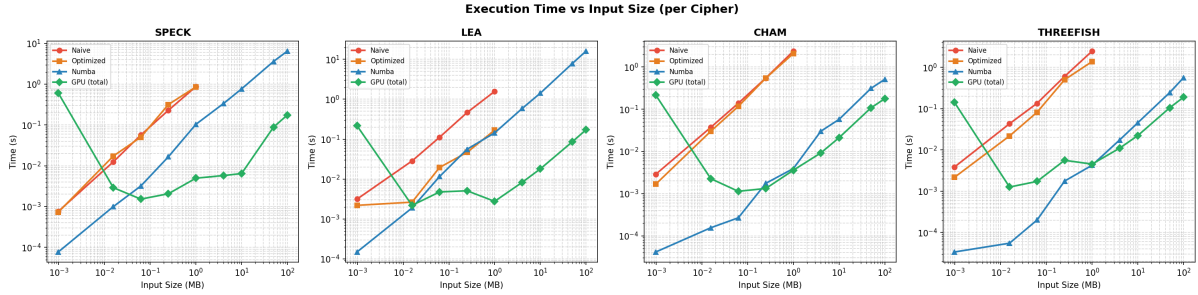


Figure 1: ARX Ciphers: Execution Time vs. Input Size across all implementation tiers.

6.2.2 GPU Speedup over Numba JIT

Figure 2 shows the GPU speedup over the Numba JIT baseline. LEA achieves the highest sustained GPU speedup (93.6 $\times$ at 100 MB) because its Numba implementation is relatively slow (~ 6 MB/s) while the GPU kernel executes efficiently. SPECK peaks at 119 $\times$ at 10 MB but drops at larger sizes due to memory transfer overhead. CHAM and Threefish show modest GPU speedups (2–3 $\times$) since their Numba implementations are already fast (~ 180 – 200 MB/s).

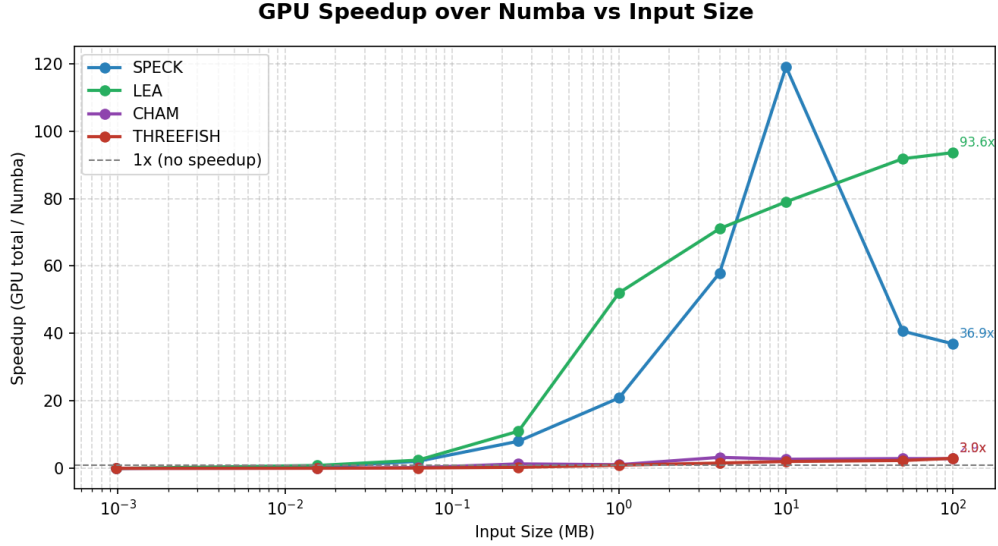


Figure 2: ARX Ciphers: GPU Speedup over Numba JIT baseline vs. Input Size.

Table 1: ARX Ciphers: GPU Speedup over Numba at Key Data Points

Cipher	Speedup @ 1 MB	Speedup @ 10 MB	Speedup @ 100 MB
SPECK	20.88×	119.04×	36.91×
LEA	52.04×	79.03×	93.61×
CHAM	1.09×	2.71×	2.87×
Threefish	0.94×	2.04×	2.97×

6.2.3 Throughput Comparison

Figure 3 shows throughput for GPU and Numba implementations. All ciphers converge to roughly 530–580 MB/s GPU throughput at large sizes, suggesting the GPU is **memory-transfer-bound**—the PCIe bus becomes the bottleneck regardless of cipher complexity.

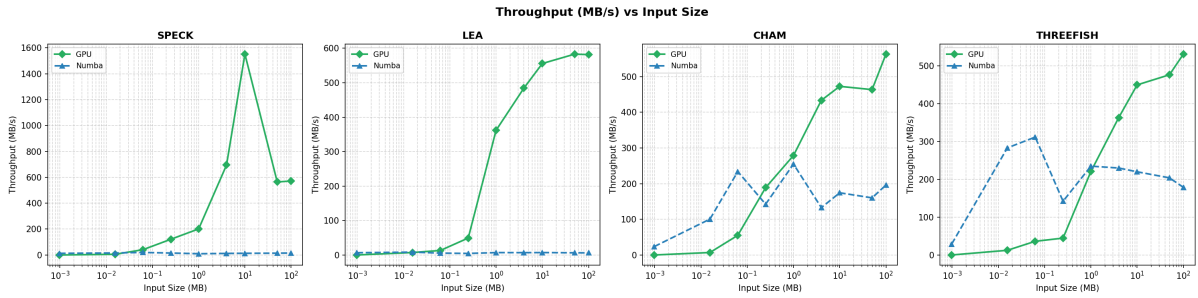


Figure 3: ARX Ciphers: Throughput (MB/s) — GPU vs. Numba JIT.

Table 2: ARX Ciphers: Throughput at 100 MB

Cipher	GPU Throughput (MB/s)	Numba Throughput (MB/s)
SPECK	571.67	15.49
LEA	581.65	6.21
CHAM	563.10	196.16
Threefish	530.83	178.73

6.2.4 GPU Time Breakdown: Kernel vs. Memory Transfer

Figure 4 decomposes GPU execution into kernel compute and memory transfer time. At 100 MB, memory transfer accounts for **78–87%** of total GPU time across all ARX ciphers. The kernel compute itself is extremely fast; the bottleneck is moving data across the PCIe bus.

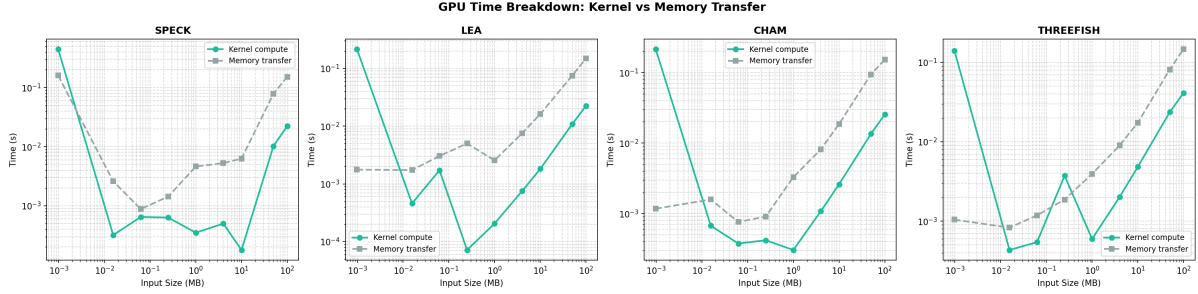


Figure 4: ARX Ciphers: GPU Time Breakdown — Kernel Compute vs. Memory Transfer.

Table 3: ARX Ciphers: GPU Time Breakdown at 100 MB

Cipher	Numba (s)	GPU Total (s)	GPU Kernel (s)	GPU Memory (s)
SPECK	6.456	0.175	0.022	0.153
LEA	16.094	0.172	0.023	0.149
CHAM	0.510	0.178	0.025	0.152
Threefish	0.560	0.188	0.041	0.147

6.2.5 Execution Time at 100 MB

Figure 5 summarizes execution time at 100 MB across all methods and ciphers. LEA Numba is the slowest CPU implementation (16 s), making GPU acceleration particularly impactful. GPU kernel times are remarkably similar across all ciphers (0.022–0.041 s), confirming massive GPU compute headroom.

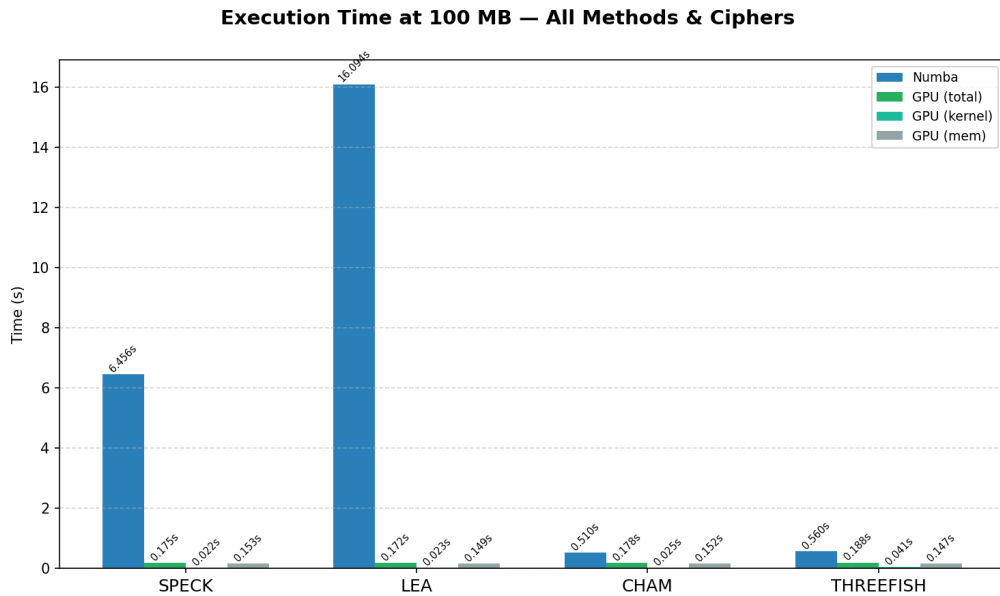


Figure 5: ARX Ciphers: Execution Time at 100 MB — All Methods and Ciphers.

6.3 Lightweight Feistel Cipher Results (SIMECK, SIMON, TWINE, WARP)

Each Feistel cipher was benchmarked across single-threaded C/C++, OpenMP multi-threaded, and CUDA GPU implementations. For TWINE and WARP, bit-sliced GPU implementations process 32 blocks per thread. Input sizes ranged from $N = 2^{10}$ to 10^8 blocks.

6.3.1 SIMECK Results

Figure 6 shows three-column results for SIMECK: execution time, throughput, and GPU speedup over CPU. SIMECK achieves a peak GPU throughput of ~ 70 GB/s and sustained speedups exceeding $800\times$ at the largest input sizes, demonstrating excellent GPU utilization due to its simple bitwise rotation-AND-XOR round structure.

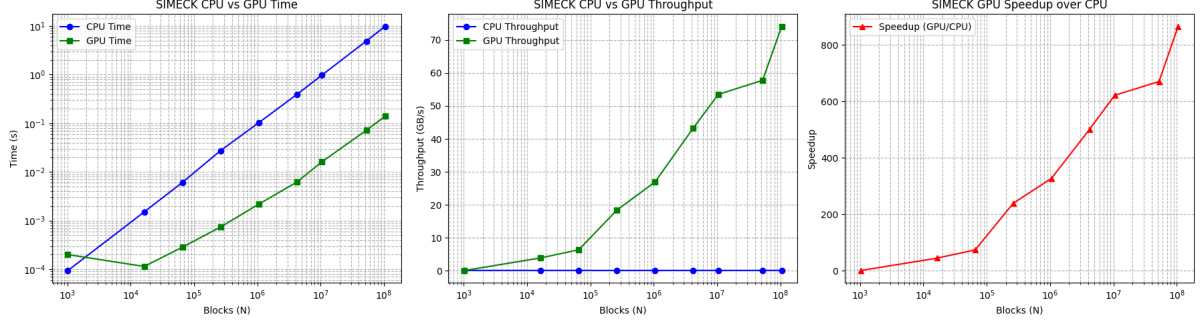


Figure 6: SIMECK: CPU vs. GPU Time, Throughput, and Speedup.

6.3.2 SIMON Results

Figure 7 shows the SIMON performance results. SIMON achieves a peak GPU throughput exceeding 40 GB/s and speedups of $\sim 550\times$ at the largest tested input sizes. Its round structure is similar to SIMECK and maps well to GPU execution.

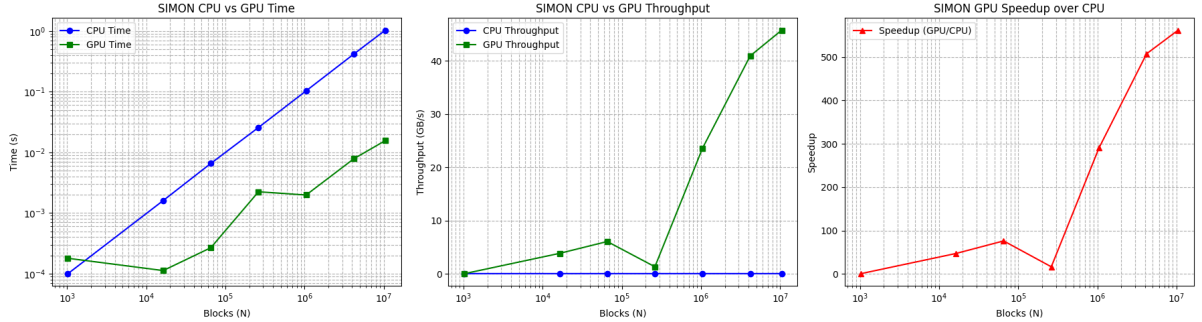


Figure 7: SIMON: CPU vs. GPU Time, Throughput, and Speedup.

6.3.3 TWINE Results (Bit-Sliced)

Figure 8 shows the TWINE performance results using the bit-sliced GPU implementation. TWINE achieves a peak GPU throughput of ~ 27 GB/s and speedups exceeding $60\times$ over the OpenMP CPU baseline. The bit-sliced approach processes 32 blocks per thread, significantly improving GPU utilization for this nibble-oriented cipher.

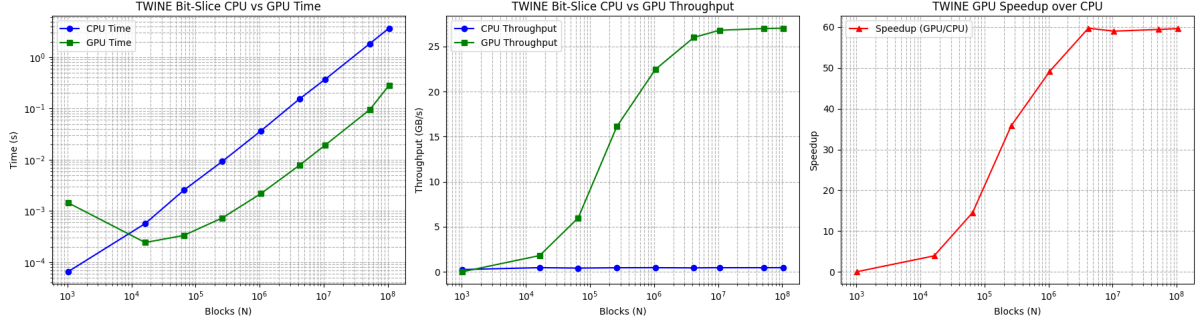


Figure 8: TWINE (Bit-Sliced): CPU vs. GPU Time, Throughput, and Speedup.

6.3.4 WARP Results (Bit-Sliced)

Figure 9 shows the WARP performance results using the bit-sliced GPU implementation. WARP achieves a peak GPU throughput of ~ 0.38 GB/s and speedups up to $7\times$ over the CPU baseline. Compared to the other Feistel ciphers, WARP shows more modest GPU speedups due to its more complex substitution-permutation structure with linear diffusion steps.

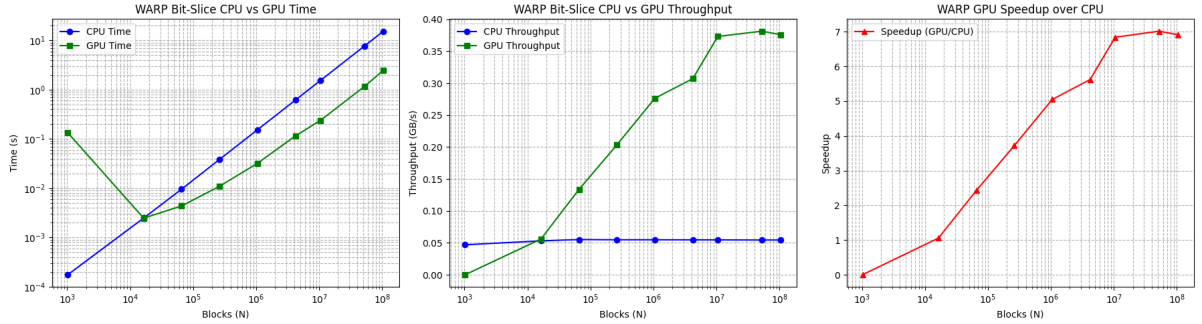


Figure 9: WARP (Bit-Sliced): CPU vs. GPU Time, Throughput, and Speedup.

6.3.5 Feistel Cipher Observations

- **SIMECK and SIMON** achieve the highest speedups ($550\text{--}800\times$) among all twelve ciphers, owing to their extremely simple round functions (rotation + AND + XOR) which map directly to fast GPU bitwise instructions with minimal register pressure.
- **TWINE bit-sliced** achieves strong speedups ($\sim 60\times$) by processing 32 blocks per thread, effectively leveraging bit-level parallelism on the GPU.
- **WARP bit-sliced** shows more moderate speedups ($\sim 7\times$) due to its more complex linear diffusion layer.
- GPU throughput scales consistently with input size for all Feistel ciphers until PCIe transfer saturation occurs at very large sizes.
- Memory transfer dominates performance for small inputs across all ciphers.

6.4 SPN Cipher Results (AES, GIFT, SKINNY, PRESENT)

6.4.1 ECB Mode: Cross-Cipher GPU Throughput

Figure 10 shows GPU vs. CPU throughput for all four SPN ciphers in ECB mode. AES dominates absolute throughput at every block size, while SKINNY bitsliced is the fastest lightweight cipher.

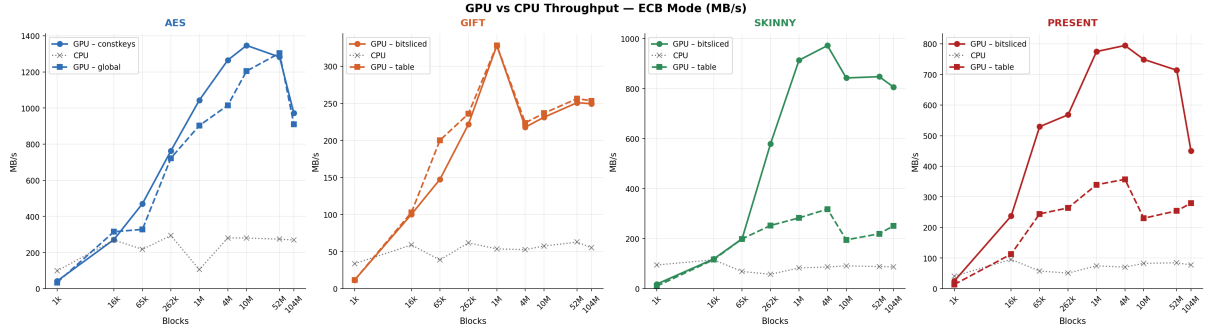


Figure 10: SPN Ciphers: GPU vs. CPU Throughput in ECB Mode (MB/s).

Table 4: SPN Ciphers ECB Mode: GPU Throughput (MB/s) — Best Variant

Blocks	AES (constkeys)	GIFT (table)	SKINNY (bitsliced)	PRESENT (bitsliced)
1,024	42.2	11.7	18.3	24.9
65,536	470.4	200.2	199.8	529.4
1,048,576	1,043.5	328.3	913.3	774.9
4,194,304	1,264.8	223.7	971.8	794.6
10,485,760	1,346.5	236.8	842.4	749.3
104,857,600	972.4	253.5	806.1	449.9

Observations:

- AES dominates absolute throughput (peaking at $\sim 1,346$ MB/s at 10.5M blocks) because its 128-bit block size moves $2\times$ more data per operation than the 64-bit ciphers.
- Bitsliced variants drastically outperform table lookups for lightweight ciphers. SKINNY bitsliced peaks at 972 MB/s, nearly matching AES.
- At extreme sizes (100M+ blocks), all ciphers experience throughput degradation due to PCIe bandwidth saturation.

6.4.2 ECB Mode: Speedup Evaluation

Figure 11 shows GPU speedup over the compiled CPU baseline for SPN ciphers.

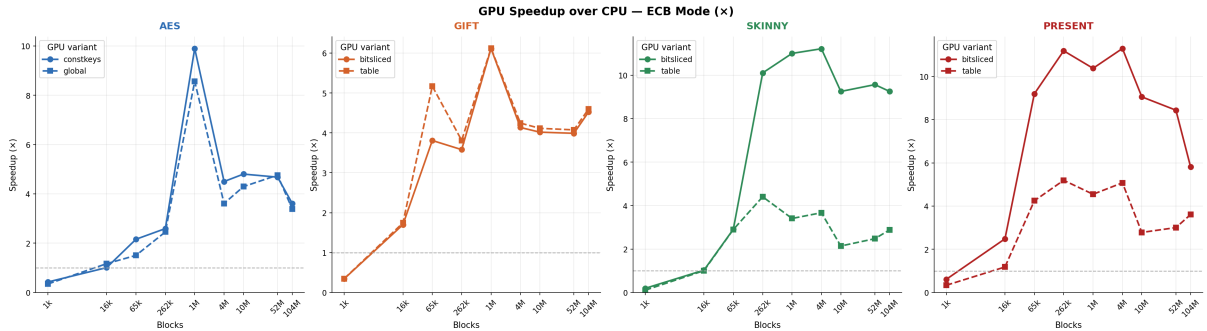


Figure 11: SPN Ciphers: GPU Speedup over CPU in ECB Mode.

Table 5: SPN Ciphers ECB Mode: GPU Speedup over Compiled CPU Baseline

Blocks	AES (constkeys)	GIFT (table)	SKINNY (bitsliced)	PRESENT (bitsliced)
16,384	1.01×	1.75×	1.03×	2.49×
262,144	2.59×	3.81×	10.11×	11.19×
4,194,304	4.50×	4.25×	11.22×	11.29×
104,857,600	3.61×	4.60×	9.25×	5.81×

Observations:

- **Break-even point:** GPU execution overtakes CPU between 16K and 65K blocks. Below this, CUDA kernel launch and transfer latency exceed execution time.
- **Peak Speedups:** PRESENT and SKINNY maintain $\sim 11\times$ speedups across a wide range (262K–4M blocks). AES peaks at $9.90\times$ at 1M blocks.
- **AES speedup is lower** than SKINNY/PRESENT despite higher throughput because the AES CPU baseline is already fast (~ 270 MB/s).

6.4.3 CTR Mode Performance

Figure 12 shows CTR mode speedups. PRESENT achieves a massive **14.21×** speedup in CTR mode at 1M blocks thanks to the native fused kernel that bypasses host-side XOR bottlenecks.

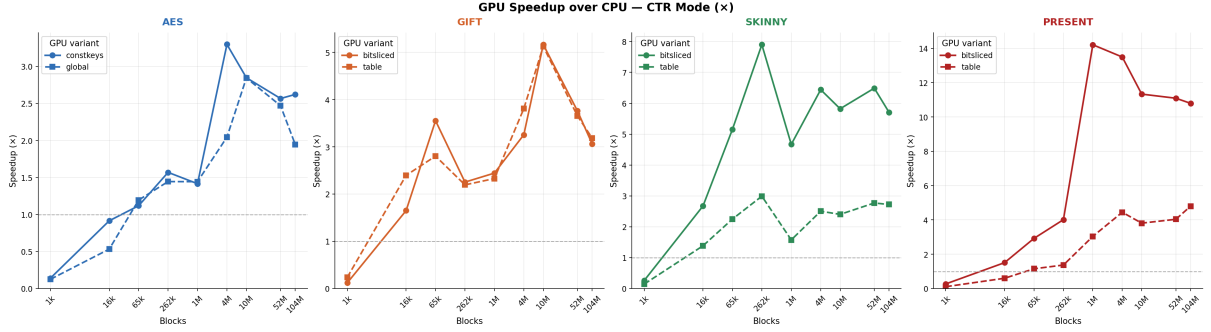


Figure 12: SPN Ciphers: GPU Speedup over CPU in CTR Mode.

6.4.4 Cross-Cipher Summary

Figure 13 provides a 2×2 grid comparing best-variant throughput and speedup across ECB and CTR modes for all SPN ciphers.



Figure 13: SPN Ciphers: Cross-Cipher Best-Variant Comparison (Throughput and Speedup, ECB and CTR).

6.4.5 Transfer Overhead Analysis

To understand true GPU efficiency, we isolated kernel execution time from memory transfer (H2D + D2H) time. At sizes above 4M blocks, **AES is 90% transfer-dominated**. Its kernel computation represents a $170\times$ speedup over the CPU, but moving 16-byte blocks over PCIe drags the end-to-end speedup down to $\sim 4.5\times$. In contrast, GIFT remains compute-dominated (only 28% transfer time), while SKINNY and PRESENT are moderately transfer-dominated at 70–82%.

Table 6: SPN Ciphers: Transfer Overhead Percentage (ECB, Best Variant)

Blocks	AES	GIFT	SKINNY	PRESENT
65,536	81%	49%	80%	75%
262,144	89%	52%	81%	82%
1,048,576	88%	43%	81%	82%
4,194,304	90%	28%	81%	82%
104,857,600	87%	39%	73%	56%

6.4.6 SPN Peak Performance Summary

Table 7: SPN Ciphers: Peak Performance Summary

Metric	Winner	Value	Blocks	Mode
Highest GPU throughput	AES (constkeys)	1,346 MB/s	10.5M	ECB
Highest ECB speedup	PRESENT (bitsliced)	11.29×	4M	ECB
Highest CTR speedup	PRESENT (bitsliced)	14.21×	1M	CTR
Highest kernel-only speedup	AES (constkeys)	170.9×	1M	ECB
Fastest lightweight cipher	SKINNY (bitsliced)	972 MB/s	4M	ECB
Most transfer-dominated	AES	92%	52M	ECB
Most compute-dominated	GIFT (table)	72% compute	10.5M	ECB

6.5 Cross-Family Analysis

Across all three cipher families, several common trends emerge:

- **PCIe Transfer is the Universal Bottleneck:** For ARX ciphers, memory transfer accounts for 78–87% of GPU time at 100 MB. For SPN ciphers, AES reaches 90% transfer-dominated at 4M blocks. All cipher families converge to PCIe bandwidth limits at large input sizes.
- **Simple Round Functions Yield Highest Speedups:** SIMECK and SIMON (simple rotation-AND-XOR) achieve 550–800× speedups. In comparison, SPN ciphers peak at ~11× (against optimized Numba baselines) and ARX ciphers peak at ~93× (LEA, against Numba).
- **Bit-Slicing is Highly Effective on GPU:** TWINE and WARP (Feistel family), SKINNY and PRESENT (SPN family) all benefit significantly from bit-sliced implementations, achieving 3–12× better kernel performance than table-based approaches.
- **Speedup Depends on CPU Baseline Quality:** Ciphers with slow CPU implementations (LEA, PRESENT pLayer) see proportionally higher speedups. Fast CPU baselines (AES JIT ~270 MB/s, CHAM Numba ~196 MB/s) limit the observable speedup ratio.

7 Conclusion

This project demonstrates that GPU acceleration significantly improves the performance of block cipher encryption across ARX, Feistel, and SPN architectures.

Key takeaways include:

- **Massive Speedups for Simple Feistel Ciphers:** SIMECK and SIMON achieve the highest speedups overall (550–800×), demonstrating that simple bitwise operations map exceptionally well to GPU hardware.
- **Domination of Memory Transfer:** GPU time is shown to be dominated by memory transfer, so we see that ciphers with slower Numba implementations see proportionally higher speedups. Faster Numba code is seen due to simple ARX operations as seen in the point below.
- **Effective ARX Acceleration:** LEA achieves 93× sustained speedup and SPECK peaks at 119×, while all ARX ciphers converge to ~550 MB/s GPU throughput at scale.
- **Importance of Architecture:** Bitsliced S-boxes and delta-swap permutation layers drastically outperform standard memory-table lookups on GPUs. SKINNY bitsliced achieves 972 MB/s, nearly matching AES’s 1,346 MB/s.
- **Transfer Limits:** For highly optimized algorithms like AES and bitsliced SKINNY, PCIe bandwidth is the primary bottleneck, with memory transfer accounting for up to 90% of total execution time at large payload sizes. At large memory amounts, transfer time becomes nearly identical and the kernel does not change a lot.

- **Fused Operations:** Fusing counter generation and XOR operations directly into the GPU kernel (as seen in PRESENT's native CTR mode achieving $14.21\times$ speedup) is crucial to preventing host-side bottlenecks.

Future work includes exploring asynchronous data transfers (CUDA streams), overlapping computation with memory transfer using pinned memory, and extending bitsliced architectures to remaining ciphers.

8 Instruction to reproduce results

- SKINNY, AES, GIFT, PRESENT - Re-run the verify and benchmark files for each cipher
- LEA, SPECK, CHAM, Three fish - run the main.py, and install the requirements.txt
- SIMECK, SIMON, TWINE, WARP - Run either of the exe files or, Install CUDA 12.3, MS VS 2022, MSVC V143 and recompile the code with -O3 for each of the .cpp and .cu file respectively with g++ and nvcc.